AssetMon — Deployment Plan

Deployment process, pricing by userbase, pros & cons, and current concerns for all four hosting options.

AssetMon is a multi-tenant SaaS (Next.js front end, NestJS back end, PostgreSQL database) where every customer gets their own isolated database. The deployment logic is the same for every option — **strong secrets → a database allowed to create new databases → build with the correct API URL → migrate → create the first admin → back up** — only the tooling differs. The app currently runs as a **single back-end instance**, so all options handle a similar number of logged-in users until some scaling code changes are made.

The Four Options — Deployment in Detail

Option A — Single Server + Docker

RECOMMENDED

Rent one cloud server and run the app as four containers: website (Next.js), back end (NestJS), database (PostgreSQL) and a secure gateway (Caddy) for HTTPS. Docker packages each part so the whole stack starts with one command; data and uploads live on storage volumes that survive restarts.

Deployment process

1. Provision an Ubuntu VM; point the domain's DNS to it; open ports 80 and 443 only.
2. Install Docker and the Compose plugin (the only software installed directly).
3. Add the deployment files: Dockerfiles (api, web), docker-compose.yml, Caddyfile, and a production .env with strong secrets and NODE_ENV=production.
4. Build and start everything: `docker compose up -d --build` (runs the Prisma client generation, back-end and front-end builds; the website's API URL is set at build time).
5. Create and migrate the catalog database: `docker compose run --rm api npm run db:catalog:migrate`. (Per-customer databases are then created automatically on each signup.)
6. Create the first admin: `...platform:bootstrap-admin -- --allow-prod`.
7. Smoke-test (sign up a test workspace, confirm its database is created, log in).
8. Set a daily off-site database backup and an uptime check. Updates later = `git pull` + rebuild.

Cost: ~$10-20 / month	**Handles:** tens of active users, 100s of accounts
Setup effort: Medium (one-time, then repeatable)	**Best for:** Launch / MVP / staging

Pros	Cons
• Low cost and fully reproducible — easy to move or rebuild • Fits the app's single-instance design; full control of the database • Database requirement (create-new-database) is satisfied automatically by the Postgres container • Simple to back up	• You manage the server, updates, and backups • Single machine — a failure means downtime until restored • Scaling up is a manual step

Option B — Single Server (Manual, no Docker)

The same single server, but each part is installed directly onto the operating system. The back end and website run as background services; PostgreSQL and the gateway are installed natively. Leanest setup, but harder to reproduce or move later.

Deployment process

1. Provision an Ubuntu VM; point DNS; configure the firewall.
2. Install Node.js 20, PostgreSQL 16, and Caddy.

3. In PostgreSQL, create a database role with the create-database privilege, set a strong password, and create the catalog database.
4. Clone the repo, install dependencies, generate the Prisma clients, run the catalog migration, and build both apps.
5. Configure the production .env (strong secrets, a real uploads folder path, NODE_ENV=production).
6. Create background services for the API and website so they auto-start and auto-restart.
7. Configure Caddy as the reverse proxy with automatic HTTPS; create the first admin; smoke-test.
8. Set daily off-site database backups. Updates later = pull, rebuild, restart services.

Cost: ~$5-15 / month (cheapest)	**Handles:** same as Option A (same hardware)
Setup effort: Medium-High (more manual steps)	**Best for:** Tight budgets / simple demos

Pros	Cons
• Lowest monthly cost; leanest use of resources • Direct, transparent control of every component • No Docker layer to learn or maintain	• Hard to rebuild or migrate — everything is hand-configured • More room for setup mistakes • You manage everything; same single-machine risk as A

Option C — Managed Platform (Railway / Fly.io)

Hand the code to a managed platform that runs and maintains the servers for you. The website and back end run as managed services and the platform provides the database. Scaling is a setting change rather than a rebuild — a strong fit once the product grows.

Deployment process

1. Create a project on the platform and add its PostgreSQL database (confirm it allows creating new databases — Railway and Fly.io do).
2. Add the back-end service from the repo (via a Dockerfile); keep it at a single instance; set the secrets and database URLs.
3. Add the website service; set its API URL (build-time) to the back end's public address.
4. Attach a storage volume to the back end for uploaded files.
5. Run the catalog migration and create the first admin using the platform's one-off command runner.
6. Connect your custom domain; the platform issues HTTPS automatically.
7. Smoke-test. Updates later happen automatically on each git push.

Cost: ~$25-60 / month	**Handles:** similar users; scales up with a click
Setup effort: Medium (no server admin)	**Best for:** A growing product, small team

Pros	Cons
• No server maintenance, patching, or uptime management • Effortless to scale up; managed add-ons (e.g. Redis) easy to attach later • Automatic deploys on every code push	• Higher monthly cost • Must confirm the database allows per-customer database creation (many cheaper managed databases do not) • Less low-level control; some vendor lock-in

Option D — Split (Vercel + Server / Database)

Host the public website on Vercel (a global, fast, SEO-friendly network) while the back end and database run on a server or managed host (as in Option A or C). Excellent marketing-site performance, at the cost of running and linking two environments.

Deployment process

1. Deploy the back end and database first using Option A or C, ending with a stable public API address.
2. Create a Vercel project from the repo pointing at the web app; set its API URL and site URL (build-time) to the live addresses.
3. Add the Vercel website domain to the back end's allowed-origins (CORS) list.
4. Vercel builds and deploys the website automatically and provides HTTPS and a global CDN.
5. Create the first admin on the back end; smoke-test (note: only fixed domains are allowed by CORS, so temporary preview URLs are blocked).
6. Back end updates use its own method; the website redeploys automatically on each git push.

Cost: ~$30-60 / month (two providers)	**Handles:** huge visitor traffic; users similar to others
Setup effort: High (two environments + linking)	**Best for:** Marketing / SEO-led growth

Pros	Cons
• Best performance and SEO for public pages — drives sign-ups • The website scales automatically with traffic • Clean separation of public site from the application	• Most complex setup; two providers and two bills • Extra secure-linking (CORS) configuration between the two • Logged-in-user limit is still set by the single back end

Pricing by Userbase (Low → Large) and Growth

Userbase	Typical option	Cost / month	Notes
Low — demo, < ~50 users	A or B	$5-20	one small server
Small — early prod, < ~500 users / dozens of customers	A (sized up) or C	$20-50	add backups + monitoring
Growing — < ~2,000 users	C or D	$50-120	bigger instance, managed add-ons
Large — 2,000+ users	Managed + code upgrades	$150+	needs scaling changes (below)

Cost grows **with** usage — no large upfront spend, and the cost *per user* falls as you grow. Reaching the “large” tier needs one-time code upgrades (shared session store, cloud file storage, database connection pooling) so the back end can run as multiple copies — required on **any** option.

What Concerns It As of Now

Concern	Why it matters	What to do
No deployment files yet	No Dockerfile / compose / CI exists — nothing is ready to run on a server	Create them once (can be done now)
Single back-end instance	In-memory state means it can't run as multiple copies — caps concurrent users	Plan scaling code upgrades before the large tier
Database-per-customer needs special rights	Needs a create-database privilege; connections grow with customer count	Self-host the DB or use Railway/Fly; add a pooler at scale
Uploads stored on local disk	Files are lost on restart and not shared across servers	Use a persistent volume now; cloud storage at scale
Secrets are still dev defaults	The app refuses to start in production with weak/default secrets	Generate strong secrets before deploying

Concern	Why it matters	What to do
Config mismatches	Web port and site URL disagree; uploads path is Windows-style	Reconcile values before deploying
No automated test suite	Only manual QA scripts exist — regressions are harder to catch	Use the existing QA scripts; add CI later

Recommendation

Start with Option A (single server + Docker): best balance of low cost, reliability, and simplicity, and a perfect fit for the current design. **Move to C or D** as usage grows — minimal rework because the app stays containerized and portable. **Before any launch:** rotate secrets, enable HTTPS, and set automated off-site backups.